

Building a Power Engineering RAG Assistant

بناء مساعد هندسي معزز بالاسترجاع

MATLAB laboratory for document retrieval, grounding, and prompt construction

لاسترجاع المستندات وبناء إجابات موثقة MATLAB مخبر

د. م. أوس غباش | إعداد: Dr.-Ing. Aouss Gabash

Laboratory Objectives

- Create a small power-engineering knowledge base.
- Preprocess and vectorize document chunks.
- Retrieve relevant passages using cosine similarity.
- Construct a grounded prompt.
- Generate a deterministic extractive answer without an external API.
- Prepare an optional interface for a connected language model.

أهداف المخبر

- إنشاء قاعدة معرفة هندسية صغيرة.
- معالجة المقاطع وتحويلها إلى متجهات.
- استرجاع المقاطع بتشابه جيب التمام.
- بناء أمر مستند إلى الأدلة.
- إنشاء إجابة استخراجية دون API خارجي.
- تهيئة الواجهة لنموذج لغوي اختياري.

1. Requirements

- MATLAB
- Text Analytics Toolbox

The laboratory uses bag-of-words TF-IDF retrieval so it remains transparent and can run without an external language-model service.

1. المتطلبات

يلزم MATLAB مع Text Analytics Toolbox. يستخدم المخبر TF-IDF حتى يعمل دون خدمة خارجية.

2. Knowledge Base

Create a file named `power_knowledge.csv` with columns ID, Title, and Text.

ID,Title,Text

```
1,Battery SOC,"State of charge is the ratio of stored energy to
2,DC Power Flow,"The DC approximation assumes voltage magnitudes
3,Voltage Stability,"Voltage stability concerns the ability of a
4,Load Forecasting,"Short-term load forecasting predicts demand
5,State Estimation,"State estimation combines redundant noisy me
6,Transformer Attention,"Self-attention relates every time step
7,GNN,"A graph neural network updates each bus representation by
8,Demand Response,"Demand response changes electricity consumpti
```

2. قاعدة المعرفة

أنشئ ملف CSV يحوي معرف المقطع وعنوانه ونصه. ويمكن توسيعه لاحقًا بصفحات المحاضرات أو كتيبات التشغيل.

3. Complete MATLAB Script

```
%% Lab 14: Transparent RAG Assistant in MATLAB
```

```
clear; clc; close all;
```

```
%% 1) Load knowledge base
```

```
KB = readtable("power_knowledge.csv", ...  
    TextType="string");
```

```
documents = tokenizedDocument(KB.Text);
```

```
%% 2) Text preprocessing
```

```
documents = lower(documents);  
documents = erasePunctuation(documents);  
documents = removeStopWords(documents);
```

```
%% 3) Build TF-IDF document representation
```

```
bag = bagOfWords(documents);  
X = tfidf(bag);
```

```
% Normalize each document vector
```

```
docNorm = sqrt(sum(X.^2,2));  
docNorm(docNorm==0) = 1;  
Xn = X./docNorm;
```

```
%% 4) User question
```

```
question = ...
```

```
    "Why are graph neural networks suitable for state estimation
```

```
queryDoc = tokenizedDocument(question);
```

```
queryDoc = lower(queryDoc);
```

```
queryDoc = erasePunctuation(queryDoc);
```

```
queryDoc = removeStopWords(queryDoc);
```

```
%% 5) Transform query into the same vocabulary
```

```
qCounts = encode(bag,queryDoc);  
idfVector = log(size(X,1)./(1+sum(X>0,1)))+1;  
q = qCounts.*idfVector;
```

```
qNorm = sqrt(sum(q.^2,2));
```

```
if qNorm==0
```

```

        error("The question contains no known vocabulary.");
    end
    qn = q/qNorm;

    %% 6) Cosine similarity
    scores = Xn*qn';

    topK = 3;
    [sortedScores,idx] = maxk(scores,topK);

    Retrieved = table( ...
        KB.ID(idx),KB.Title(idx),KB.Text(idx),sortedScores, ...
        VariableNames=["ID","Title","Text","Score"]);

    disp(Retrieved);

    %% 7) Build grounded context
    contextBlocks = strings(topK,1);

    for k = 1:topK
        contextBlocks(k) = ...
            "["+Retrieved.ID(k)+"] "+ ...
            Retrieved.Title(k)+": "+ ...
            Retrieved.Text(k);
    end

    context = join(contextBlocks,newline);

    %% 8) Construct a grounded prompt
    prompt = ...
        "You are a power-engineering teaching assistant."+newline+ ...
        "Answer only from the supplied context."+newline+ ...
        "If the context is insufficient, say so."+newline+ ...
        "Preserve engineering terminology and cite source IDs."+newline+
        "CONTEXT:"+newline+context+newline+newline+ ...
        "QUESTION:"+newline+question+newline+newline+ ...
        "ANSWER:";

    disp(prompt);

```

```

%% 9) Deterministic extractive answer
% This stage avoids an external API.
answer = ...
"Graph neural networks are suitable because they represent "+ ..
"the electrical grid as buses connected by lines and update "+ ..
"each bus using information from electrically connected "+ ...
"neighbors [7]. State estimation also uses network-wide "+ ...
"measurements to estimate voltage states [5]. Therefore, "+ ...
"the graph structure matches the physical topology used by "+ ..
"the estimation problem.";

fprintf("\nANSWER:\n%s\n",answer);

%% 10) Retrieval visualization
figure;
barh(categorical(Retrieved.Title),Retrieved.Score);
grid on;
xlabel("Cosine Similarity");
title("Top Retrieved Knowledge Chunks");

```

3. الكود الكامل

يقرأ الكود قاعدة المعرفة، ويعالج النصوص، ويبني TF-IDF، ثم يسترجع أفضل المقاطع ويبني أمرًا مستندًا إلى الأدلة.

4. TF-IDF

$$TF - IDF(t, d) = TF(t, d) \cdot \log(N/DF(t))$$

Frequent terms within one document receive weight, while terms appearing in every document are down-weighted.

4. طريقة TF-IDF

تعطي وزنًا للكلمات المهمة داخل مستند، وتخفض وزن الكلمات الشائعة في جميع المستندات.

5. Retrieval

$$\text{similarity}(q, d) = q^T d / (\|q\| \cdot \|d\|)$$

The top-k chunks become the evidence supplied to the answer-generation stage.

5. الاسترجاع

تُحسب مشابهة السؤال لكل مقطع، ثم تُختار أفضل المقاطع لتصبح الأدلة المستخدمة في الإجابة.

6. Grounded Prompt Design

The prompt explicitly instructs the assistant to:

- Use only the supplied context.
- State when evidence is insufficient.
- Preserve engineering terms.
- Cite source IDs.

6. تصميم الأمر المستند إلى المصادر

يفرض الأمر الاعتماد على السياق والتصريح عند نقص المعلومات والحفاظ على المصطلحات وذكر معرفات المصادر.

7. Optional Language-Model Connection

The retrieval stage is independent of the generation provider. A connected model can receive the variable prompt and return a generated answer.

Never place secret API keys directly inside a public HTML page or a GitHub repository. Store credentials in environment variables or a secure secret manager.

7. الربط الاختياري بنموذج لغوي

يمكن إرسال المتغير prompt إلى نموذج متصل. ويجب عدم وضع مفاتيح API في صفحة عامة أو مستودع GitHub.

8. Evaluation Questions

- Did retrieval return the correct source?
- Does the answer contain unsupported claims?
- Are citations correct?
- Are units and technical terms preserved?
- Does the system admit missing evidence?

8. أسئلة التقييم

افحص صحة الاسترجاع ووفاء الإجابة بالمصادر وصحة الإحالات والوحدات وقدرة النظام على الاعتراف بنقص الأدلة.

9. Student Experiments

1. Add 20 new engineering chunks.
2. Compare topK=1, 3, and 5.
3. Test vague and highly specific questions.
4. Add Arabic documents and questions.
5. Replace TF-IDF with document embeddings.
6. Create a confidence rule based on the best similarity score.

9. تجارب الطالب

1. أضف 20 مقطعًا.
2. قارن قيم topK.
3. اختبر أسئلة عامة ودقيقة.
4. أضف مستندات وأسئلة عربية.
5. استبدل TF-IDF بتمثيلات مستندات.
6. أنشئ قاعدة ثقة اعتمادًا على التشابه.

10. Mini Project

Course assistant: Index all lecture HTML pages, retrieve the most relevant sections, and produce bilingual answers with citations to lecture and section numbers.

10. مشروع مصغر

أنشئ مساعدًا للمقرر يفهرس جميع صفحات HTML ويجيب بالعربية والإنجليزية مع ذكر رقم المحاضرة والقسم.

11. Laboratory Report

- Describe the knowledge base.
- Explain preprocessing and TF-IDF.
- Show retrieval results for five questions.
- Evaluate one successful and one failed query.
- Discuss hallucination and source grounding.
- Propose privacy and security controls.

11. تقرير المخبر

يتضمن وصف قاعدة المعرفة والمعالجة و TF-IDF ونتائج خمسة أسئلة وحالة نجاح وفشل وتحليل الهلوسة والأمان والخصوصية.